

SQL (Sub Queries + Co-related Queries)

Introduction to Databases

SQL - Sub-Query

- A Sub-query or Inner query or a Nested query is a query within another SQL query and embedded within the WHERE clause.
- Sub-queries can be used with the SELECT, INSERT, UPDATE, and DELETE statements along with the operators like =, <, >, >=, <=, IN, BETWEEN, etc.
- A subquery's outer statement can be any one of: SELECT, INSERT, UPDATE, DELETE

SQL - Sub-Query

- There are a few rules that subqueries must follow –
- Subqueries must be enclosed within parentheses.
- A subquery can have only one column in the SELECT clause, unless multiple columns are in the main query for the subquery to compare its selected columns.
- An ORDER BY command cannot be used in a subquery, although the main query can use an ORDER BY. The GROUP BY command can be used to perform the same function as the ORDER BY in a subquery.

SQL - Sub-Query

- Subqueries that return more than one row can only be used with multiple value operators such as the IN operator.
- The SELECT list cannot include any references to values that evaluate to a BLOB, ARRAY, CLOB, or NCLOB.
- A subquery cannot be immediately enclosed in a set function.
- The BETWEEN operator cannot be used with a subquery. However, the BETWEEN operator can be used within the subquery.

SQL - Sub-Query

- A subquery is a SELECT statement that is nested within another SELECT statement and which return intermediate results.
- Syntax

SELECT column_name [, column_name]

FROM table1 [, table2]

WHERE column_name OPERATOR (SELECT column_name [,
column_name] FROM table1 [, table2] [WHERE])

Example:

STUDENT

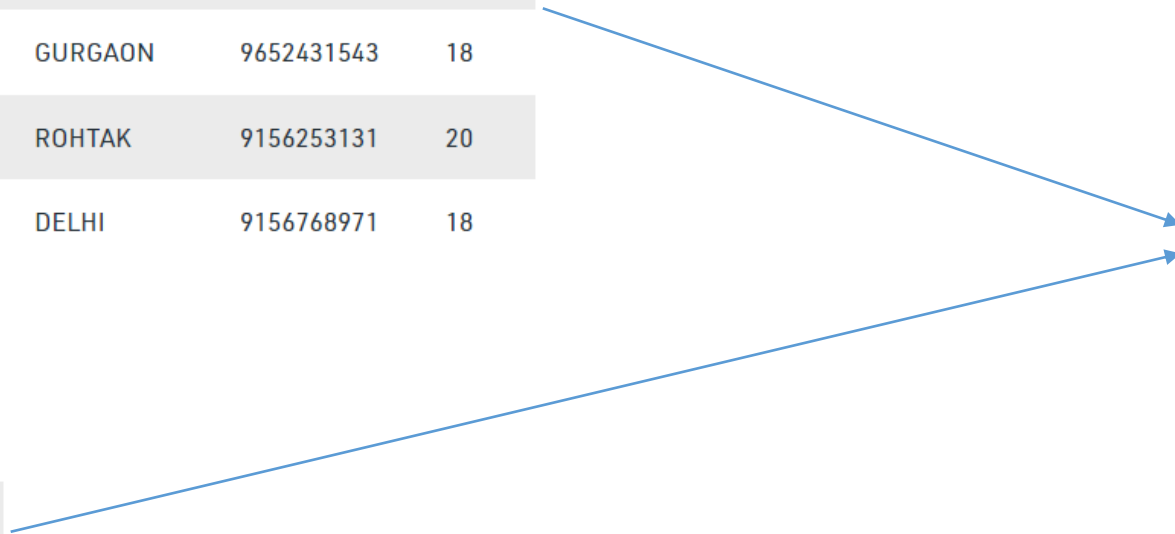
S_ID	S_NAME	S_ADDRESS	S_PHONE	S_AGE
S1	RAM	DELHI	9455123451	18
S2	RAMESH	GURGAON	9652431543	18
S3	SUJIT	ROHTAK	9156253131	20
S4	SURESH	DELHI	9156768971	18

COURSE

C_ID	C_NAME
C1	DSA
C2	Programming
C3	DBMS

STUDENT_COURSE

S_ID	C_ID
S1	C1
S1	C3
S2	C1
S3	C2
S4	C2
S4	C3



Example Query

- Select **S_ID** from **STUDENT_COURSE** where **C_ID** IN
- (SELECT **C_ID** from **COURSE** where **C_NAME** = 'DSA' or **C_NAME**='DBMS');

Example

- **STUDENT**s who have either enrolled in 'DSA' or 'DBMS', it can be done as:

- Query

```
Select S_NAME from STUDENT where S_ID IN  
(Select S_ID from STUDENT_COURSE where C_ID IN  
(SELECT C_ID from COURSE where C_NAME='DSA'  
or C_NAME='DBMS'));
```


Example

- **STUDENT**s who have neither enrolled in 'DSA' nor in 'DBMS', it can be done as:

- Solution

```
Select S_ID from STUDENT where S_ID NOT IN  
(Select S_ID from STUDENT_COURSE where C_ID IN  
(SELECT C_ID from COURSE where C_NAME='DSA'  
or C_NAME='DBMS'));
```

Subquery in From Clause

- When you use a subquery in the FROM clause, the result set returned from a subquery is used as a temporary table.
- This table is referred to as a derived table or materialized subquery.
- The following subquery finds the maximum, minimum, and average number of items in sale orders:

Subquery in From Clause - Example

```
SELECT
    MAX(items),
    MIN(items),
    FLOOR(AVG(items))
FROM
    (SELECT
        orderNumber, COUNT(orderNumber) AS items
    FROM
        orderdetails
    GROUP BY orderNumber) AS lineitems;
```

MySQL IN Operator

- The In Operator will provide you option to specify multiple values in where clause.
- It is also used for multiple OR conditions

```
SELECT column_name(s)  
FROM table_name  
WHERE column_name IN (value1, value2, ...);
```

IN Operator in Subquery

- `SELECT column_name(s)`
`FROM table_name`
`WHERE column_name IN (SELECT STATEMENT);`

Example

- `SELECT officeCode, city, phone, country FROM offices WHERE country IN ('USA' , 'France');`
- `SELECT officeCode, city, phone, country FROM offices WHERE country NOT IN ('USA' , 'France');`
- `SELECT * FROM Customers WHERE Country NOT IN ('Germany', 'France', 'UK');`

ANY and ALL Operators

- ANY and ALL operators allow you to perform a comparison between a single column value and a range of other values.
- The ANY operator:
 - returns a boolean value as a result
 - returns TRUE if ANY of the subquery values meet the condition
 - ANY means that the condition will be true if the operation is true for any of the values in the range.

ANY operator - Syntax

- *SELECT column_name(s)*
FROM table_name
WHERE column_name operator ANY
(SELECT column_name
FROM table_name
WHERE condition);

ANY operator - Example

```
SELECT Name, City FROM Customers  
WHERE CustomerNum =  
ANY (SELECT CustomerNum FROM Orders  
WHERE STATUS = 'Success');
```

All operator

- The ALL operator:
 - returns a boolean value as a result
 - returns TRUE if ALL of the subquery values meet the condition
 - is used with SELECT, WHERE and HAVING statements
 - ALL means that the condition will be true only if the operation is true for all values in the range.

ALL operator - Syntax

- `SELECT column_name(s)`
`FROM table_name`
`WHERE column_name operator ALL`
`(SELECT column_name`
`FROM table_name`
`WHERE condition);`

ALL operator - Example

```
SELECT ProductName FROM Products  
WHERE ProductID =  
ALL (SELECT ProductId FROM OrderDetails  
WHERE Quantity = 6 OR Quantity = 2);
```

EXISTS Operator

- The EXISTS operator is used to test for the existence of any record in a subquery.
- The EXISTS operator returns TRUE if the subquery returns one or more records.
- It is used in combination with a subquery and checks the existence of data in a subquery
- It means if a subquery returns any record, this operator returns true. Otherwise, it will return false.
- We can use it with SELECT, UPDATE, DELETE, INSERT statement.

EXISTS Syntax

- SELECT *column_name(s)*
FROM *table_name*
WHERE [NOT] EXISTS
(SELECT *column_name* FROM *table_name* WHERE *condition*);

EXISTS Examples

- **SELECT** SupplierName
FROM Suppliers
WHERE EXISTS (**SELECT** ProductName **FROM** Products **WHERE** Products.SupplierID = Suppliers.supplierID AND Price < 20);

SELECT name, occupation **FROM** customer
WHERE EXISTS (**SELECT** * **FROM** Orders
WHERE customer.cust_id = Orders.cust_id);

Deleting records with subqueries

- A SELECT statement within another SELECT statement can be used as a subquery along with SQL DELETE command can be used to perform a deletion.

- Example:

```
DELETE FROM customer1  
WHERE agent_code =  
ANY( SELECT agent_code FROM agents  
WHERE working_area='London');
```


Inserting records with subqueries

- The INSERT INTO SELECT statement copies data from one table and inserts it into another table.
- The INSERT INTO SELECT statement requires that the data types in source and target tables matches.
- The existing records in the target table are unaffected.
- Syntax
- INSERT INTO *table2* (*column1*, *column2*, *column3*, ...)
SELECT *column1*, *column2*, *column3*, ...
FROM *table1*
WHERE *condition*;

Inserting records with subqueries

- **INSERT INTO** invoice (date, client_id, amount, wine_name, order_id)
SELECT o.date,
o.client_id,
o.quantity * w.price as amount,
o.quantity || ' bottles of ' || o.wine_name,
o.order_id
FROM order o
JOIN wine w
ON w.name = o.wine_name
WHERE o.date = '2020-06-28'

update using subqueries

```
UPDATE wine w
SET stock = stock - (
    SELECT SUM (quantity)
    FROM order
    WHERE date = CURRENT_DATE
AND order.wine_name = w.name
```

update using subqueries

```
UPDATE wine w
SET stock = stock - (
    SELECT SUM (quantity)
    FROM order
    WHERE date = CURRENT_DATE AND
order.wine_name = w.name
)
WHERE w.name IN (SELECT order.wine_name FROM order
    WHERE date = CURRENT_DATE)
```

Correlated Subqueries

- Correlated subqueries are used for row-by-row processing. Each subquery is executed once for every row of the outer query.
- SQL Correlated Subqueries are used to select data from a table referenced in the outer query.
- The subquery is known as a correlated because the subquery is related to the outer query.
- In this type of queries, a table alias (also called a correlation name) must be used to specify which table reference is to be used.

Correlated Subqueries

- Uncorrelated subquery executes the subquery first and provides the value to the outer query, whereas correlated subquery references a column in the outer query and executes the subquery once for each row in the outer query.
- The alias is the pet name of a table which is brought about by putting directly after the table name in the FROM clause. This is suitable when anybody wants to obtain information from two separate tables.

Example

- `SELECT a.ord_num, a.ord_amount, a.cust_code,
a.agent_code
FROM orders a
WHERE a.agent_code=
(SELECT b.agent_code
FROM agents b
WHERE b.agent_name='Alex');`

Output:

ORD_NUM	ORD_AMOUNT	CUST_CODE	AGENT_CODE
200127	2500	C00015	A003
200100	1000	C00015	A003

```
SELECT a.ord_num,a.ord_amount,a.cust_code,  
a.agent_code FROM orders a
```

```
WHERE a.agent_code=
```

```
( SELECT b.agent_code  
FROM agents b  
WHERE b.agent_name='Alex' );
```

AGENT_CODE	AGENT_NAME
A003	Alex
A001	Subbarao
A009	Benjamin
A007	Ramasundar
A008	Alford
A011	Ravi Kumar
A010	Santakumar
A012	Lucida
A005	Anderson
A002	Mukesh

agents

```
WHERE b.agent_code='Alex'
```

AGENT_CODE
A003

results of
inner query

```
SELECT a.ord_num,a.ord_amount,a.cust_code,  
a.agent_code FROM orders a
```

```
WHERE a.agent_code='A003'
```

ORD_NUM	CUST_CODE	AGENT_CODE
200114	C00002	A008
200122	C00003	A004
200129	C00024	A006
200127	C00015	A003
200128	C00009	A002
200135	C00007	A010
200131	C00012	A012
200133	C00009	A002
200132	C00013	A013
200100	C00015	A003
200110	C00019	A010

orders

ORD_NUM	ORD_AMOUNT	CUST_CODE	AGENT_CODE
200127	25	C00015	A003
200100	10	C00015	A003

results

EXISTS with a Correlated Subquery

```
SELECT employee_id, manager_id, first_name, last_name
FROM employees a
WHERE EXISTS
  (SELECT employee_id
FROM employees b
WHERE b.manager_id = a.employee_id)
```

Output:

EMPLOYEE_ID	MANAGER_ID	FIRST_NAME	LAST_NAME
100		Steven	King
101	100	Neena	Kochhar
102	100	Lex	De Haan
103	102	Alexander	Hunold
108	101	Nancy	Greenberg
114	100	Den	Raphaely
120	100	Matthew	Weiss
121	100	Adam	Fripp
122	100	Payam	Kaufling
123	100	Shanta	Vollman
124	100	Kevin	Mourgos
145	100	John	Russell
146	100	Karen	Partners
147	100	Alberto	Errazuriz
148	100	Gerald	Cambrault
149	100	Eleni	Zlotkey
201	100	Michael	Hartstein
205	101	Shelley	Higgins

```
SELECT employee_id, manager_id, first_name,
last_name FROM employees a
```

WHERE EXISTS

```
(SELECT employee_id
FROM employees b
WHERE b.manager_id = a.employee_id);
```

FIRST_NAME	EMPLOYEE_ID	MANAGER_ID	EMPLOYEE_ID	MANAGER_ID	FIRST_NAME
Steven	100	-	100	-	Steven
Neena	101	100	101	100	Neena
Lex	102	100	102	100	Lex
Alexander	103	102	103	102	Alexander
Bruce	104	103	104	103	Bruce
David	105	103	105	103	David
Valli	106	103	106	103	Valli
Diana	107	103	107	103	Diana
Nancy	108	101	108	101	Nancy
Daniel	109	108	109	108	Daniel

employees alias b

employees alias a

EXISTS [TRUE]

```
SELECT employee_id, manager_id, first_name,
last_name FROM employees a
```

WHERE EXISTS [TRUE]

EMPLOYEE_ID	MANAGER_ID	FIRST_NAME	EMPLOYEE_ID	MANAGER_ID	FIRST_NAME
100	-	Steven	100	-	Steven
101	100	Neena	101	100	Neena
102	100	Lex	102	100	Lex
103	102	Alexander	103	102	Alexander
104	103	Bruce	108	101	Nancy
105	103	David	114	100	Den
106	103	Valli	120	100	Matth
107	103	Diana	121	100	Adam
108	101	Nancy			
109	108	Daniel			

employees alias a

results

Quote of the day



If you focus on **results,**
you will never **change.**

If you focus on **change,**
you will get **results.**

~**Jack Dixon**

